

Já. Við höldum þá áfram í **Technical Specification v1.0 — Infrastructure, Deployment & Production Architecture**. Hér er næsta heila útgáfa, byggð beint ofan á v0.1–v0.9.

Technical Specification v1.0

Infrastructure, Deployment & Production Architecture

Project: Icelandic Legal Case Management System — ILCMS

Version: 1.0

Status: Infrastructure Architecture Baseline

Date: September 2026

1. Purpose

Þetta skjal skilgreinir production infrastructure fyrir ILCMS.

Markmiðið er að kerfið sé:

- secure by design
- highly available
- auditable
- recoverable
- scalable
- containerized
- automatable
- suitable for sensitive legal information
- deployable on-premises eða í private cloud

Architecture skal ekki vera bundin við einn cloud provider.

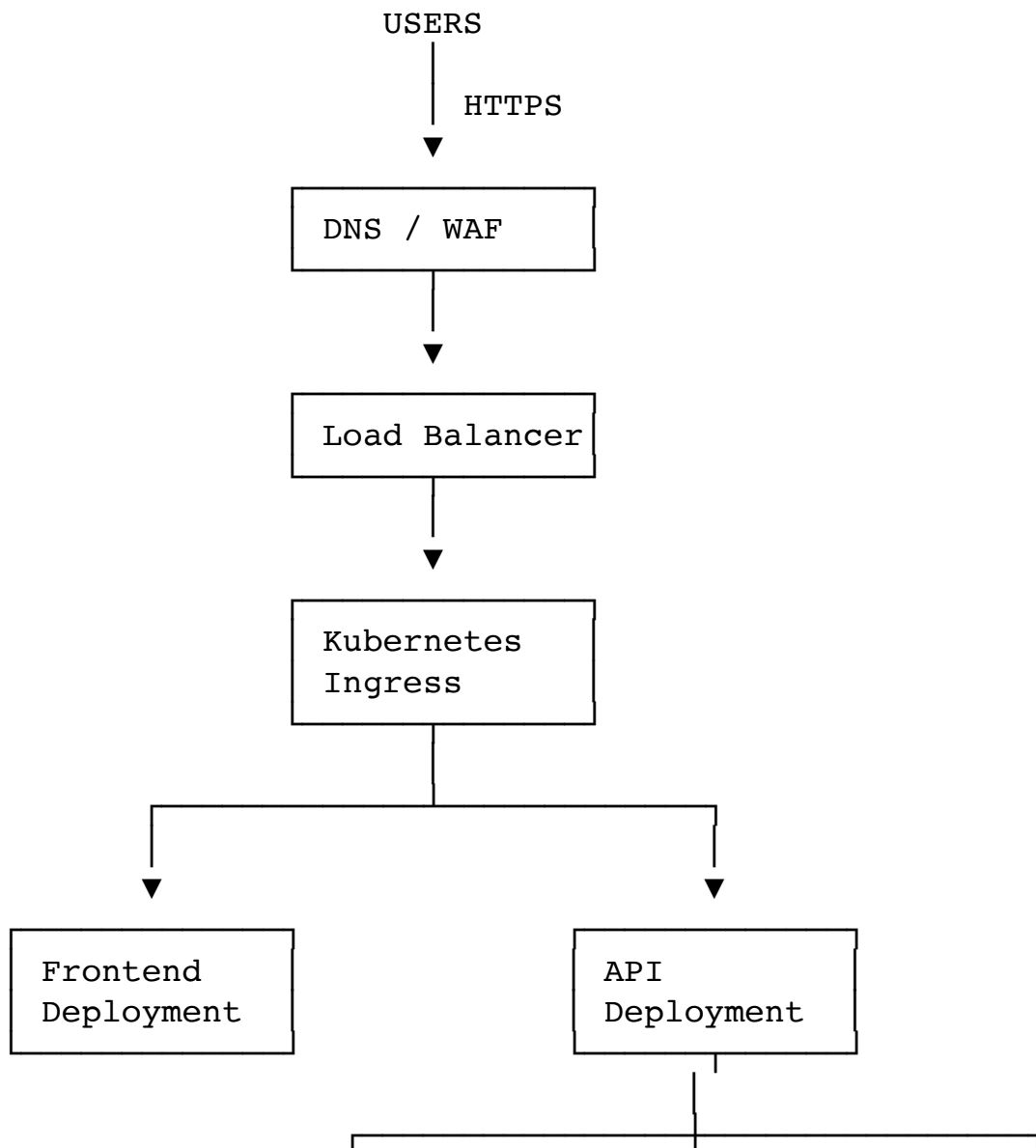
2. Infrastructure Principles

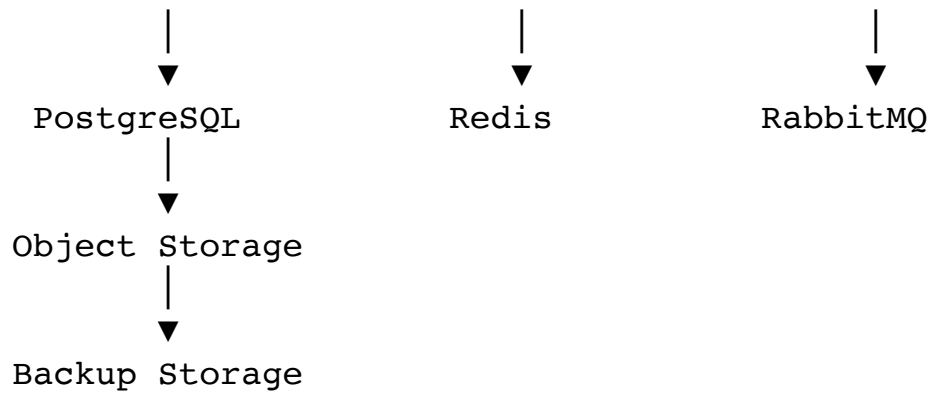
Helstu principles:

1. **Security first**
2. **Least privilege**

3. **Infrastructure as Code**
4. **Immutable deployments**
5. **Automated testing**
6. **Centralized audit**
7. **Encrypted data**
8. **Automated backup**
9. **Tested recovery**
10. **No manual production configuration where avoidable**

3. Target Architecture





4. Deployment Model

Primary target:

Kubernetes-based deployment.

Supported environments:

development

test

staging

production

Optional:

DR / disaster recovery

5. Environment Isolation

Each environment shall have:

- separate namespace
- separate secrets
- separate database
- separate object storage
- separate message queues
- separate service accounts

Production data shall never be copied into development without explicit anonymization.

6. Kubernetes

Recommended logical namespaces:

ilcms-dev
ilcms-test
ilcms-staging
ilcms-prod

Infrastructure services may use separate namespaces:

ingress-system
monitoring
logging
security

7. Kubernetes Workloads

Production:

ilcms-frontend
ilcms-api
ilcms-worker
ilcms-scheduler

Optional:

ilcms-indexer
ilcms-document-processor
ilcms-ai-worker

8. Frontend Deployment

Frontend shall be built into an immutable container image.

Example:

Browser



Ingress



Frontend Service



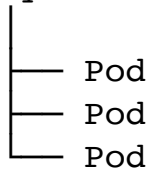
Nginx / static web server

No application secrets shall be embedded into frontend images.

9. API Deployment

API:

Deployment



Minimum production baseline:

`replicas >= 2`
for high availability.

10. Worker Architecture

Long-running/background tasks skulu ekki keyra inni í API process.

Workers sjá um:

document processing
virus scanning orchestration
OCR
search indexing
notifications
AI processing
scheduled jobs

11. Scheduler

Scheduler sér um:

deadline notifications
retention processing
scheduled jobs
maintenance tasks
periodic reconciliation
Scheduler skal tryggja að sama job keyri ekki tvisvar óvart.

12. PostgreSQL

PostgreSQL er primary relational database.

Stores:

`cases`

parties
users
documents metadata
evidence
filings
deadlines
tasks
audit metadata
workflow state

Binary documents skulu ekki vera geymd í PostgreSQL nema sérstaklega sé ákveðið annað.

13. PostgreSQL HA

Production ætti að nota:

Primary
├── synchronous replica
└── asynchronous replica / DR

HA implementation getur verið:

Patroni
CloudNativePG
managed PostgreSQL
Architecture skal ekki hard-code-a einn provider.

14. PostgreSQL Storage

Database storage skal vera:

encrypted at rest
persistent
backed up
monitored
Separate volumes:

database data
WAL
backup staging
þar sem við á.

15. Database Connections

API pods skulu ekki opna ótakmarkaðan fjölda connections.

Use:

connection pooling
t.d.:

PgBouncer
ef scale krefst þess.

16. Database Migrations

Migration tool skal vera version-controlled.

Flow:

```
Git
↓
migration
↓
CI validation
↓
staging
↓
production
```

Production migration má ekki vera handvirk SQL breyting nema emergency procedure.

17. Object Storage

Documents skulu geymd í object storage.

Logical structure:

```
ilcms/
├── documents/
├── evidence/
├── filings/
└── exports/
```

Storage þarf að styðja:

encryption
versioning
retention
immutability / object lock
access logging

18. Document Integrity

Við upload skal reikna hash.

Recommended:

SHA-256

Database geymir:

document_id

version

sha256

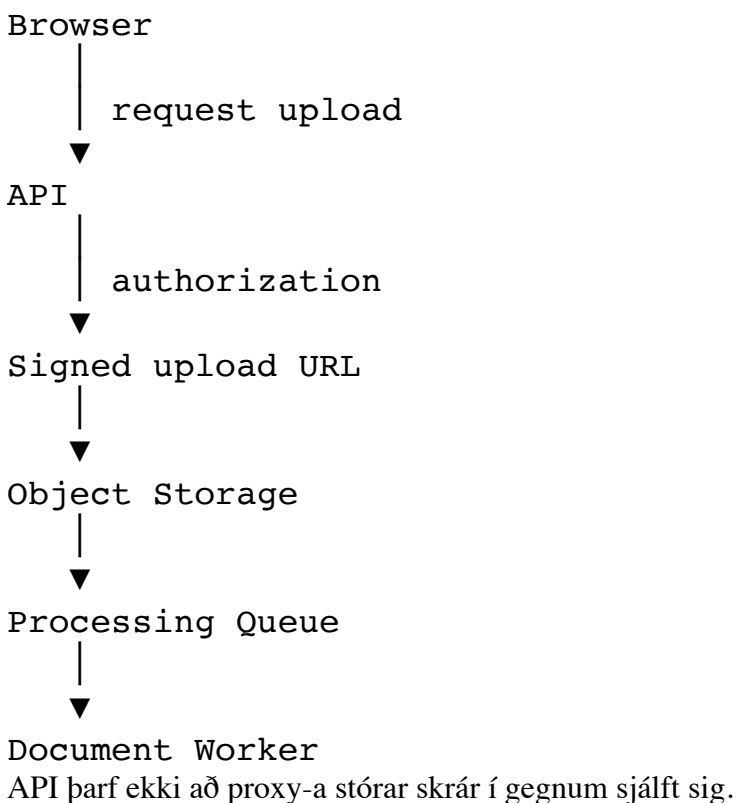
size

mime_type

storage_key

Hash er notað til integrity verification.

19. Document Upload Architecture



20. Document Processing

Processing pipeline:

Uploaded
↓
Virus Scan
↓
File Validation
↓
Metadata Extraction
↓
OCR (if required)
↓
Text Extraction
↓
Indexing
↓
Available
Ef processing mistekst:

PROCESSING_FAILED
og notandi fær skýra stöðu.

21. Antivirus

Öll uploaded files skulu fara í malware scanning.

Possible architecture:

Object Storage
↓
Queue
↓
Scanner
↓
Clean / Quarantine

Untrusted document má ekki verða available fyrir almenna notkun fyrr en scanning hefur staðfest það.

22. RabbitMQ

RabbitMQ er messaging backbone fyrir asynchronous processing.

Queues:

document.processing
document.scanning

document.indexing
notifications
ai.processing
audit.events
Queues skulu hafa:

retry
dead-letter queue
visibility/recovery strategy

23. Redis

Redis má nota fyrir:

cache
rate limiting
distributed locks
temporary state
Redis má ekki vera authoritative storage fyrir legal records.

24. Search

MVP getur notað PostgreSQL full-text search.

Architecture skal hins vegar gera mögulegt að bæta við:

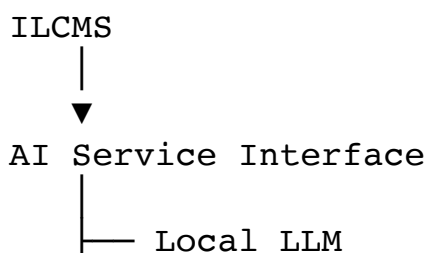
OpenSearch
Elasticsearch
síðar.

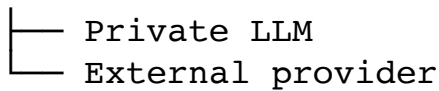
Search index er derived data.

Database er source of truth.

25. AI Infrastructure

AI services skulu vera modular.





Core application má ekki vera tightly coupled við einn AI vendor.

26. Sensitive AI Data

Legal documents skulu ekki fara til external AI provider nema:

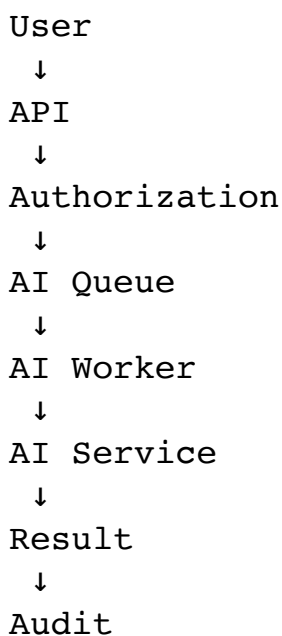
- explicit policy leyfi það
- contractual controls séu til staðar
- data protection requirements séu uppfylltar
- user/system authorization sé til staðar

Default architecture:

Private/local AI preferred.

27. AI Queue

AI jobs:



AI processing má ekki block-a HTTP request.

28. Secrets Management

Secrets skulu ekki vera:

Git
Docker image
frontend bundle
plain Kubernetes manifests
Recommended:

HashiCorp Vault
eða equivalent enterprise secret manager.

29. Kubernetes Secrets

Kubernetes Secrets skulu helst vera aðeins temporary integration layer.

Preferred:

Vault
↓
External Secrets
↓
Kubernetes workload
Secrets skulu ekki vera manually committed base64 values.

30. TLS

All external traffic:

HTTPS
TLS 1.2+
TLS 1.3 preferred.

Certificate management skal vera automated.

31. Internal TLS

Production environment ætti að styðja encryption in transit milli critical services.

Possible:

mTLS
ef security requirements justify það.

32. Network Segmentation

Logical zones:

Internet
|
DMZ
|
Application
|
Data

Database má ekki vera reachable frá Internet.

33. Network Policy

Kubernetes NetworkPolicies skulu restrict-a communication.

Example:

Frontend → API
API → PostgreSQL
API → Redis
API → RabbitMQ
Worker → Object Storage
Worker → RabbitMQ
Frontend má ekki tala beint við PostgreSQL.

34. Ingress

Ingress handles:

TLS termination
routing
rate limiting
security headers

Possible:

NGINX Ingress
Traefik
HAProxy
cloud-native ingress

35. WAF

WAF should protect public endpoints against:

SQL injection
XSS
malicious payloads
automated abuse
common OWASP attacks
WAF er additional layer, ekki substitute fyrir secure application.

36. Rate Limiting

Rate limiting skal vera sérstaklega á:

login
password recovery
search
file upload initiation
AI endpoints
public API
Internal authenticated APIs geta haft hærri limits.

37. Container Security

Images skulu:

use minimal base image
run as non-root
have read-only filesystem where practical
drop Linux capabilities
use resource limits
No privileged containers nema sérstaklega réttlætt.

38. Image Scanning

CI skal keyra vulnerability scanning.

Pipeline:

Build
↓
Dependency scan
↓
Container scan
↓
SBOM

↓
Sign image
↓
Push registry

39. Software Supply Chain

Production images skulu vera:

versioned
signed
traceable
Recommended:

SBOM
image signing
provenance metadata

40. Container Registry

Private registry:

GitLab Registry
Harbor
cloud registry
Production cluster má aðeins pull-a frá trusted registry.

41. CI/CD

Recommended pipeline:

Developer
↓
Git
↓
Pull Request
↓
Lint
↓
Unit Tests
↓
Security Scan
↓
Build

↓
Container Scan
↓
Integration Tests
↓
Deploy Staging
↓
E2E Tests
↓
Approval
↓
Production

42. Git Strategy

Recommended:

main
feature/*
bugfix/*
release/*
Production deployment skal byggja á immutable Git commit/tag.

43. Infrastructure as Code

All infrastructure skal vera declarative.

Recommended:

Terraform
fyrir infrastructure.

Kubernetes manifests:

Helm
Kustomize

44. GitOps

Production deployment getur notað:

Argo CD
Flux
Recommended model:

Git

↓

GitOps controller

↓

Kubernetes

Engin manual `kubectl apply` fyrir normal production deployment.

45. Deployment Strategy

Default:

Rolling deployment

fyrir backward-compatible changes.

Critical releases geta notað:

Blue/Green

Canary

46. Database Deployment Strategy

Application deployment:

expand

↓

migrate

↓

deploy

↓

contract

Forðast breaking database migrations í sama deployment step og app.

47. Health Checks

Every service:

/readiness

/liveness

API getur einnig haft:

/health

Readiness skal athuga hvort service geti tekið traffic.

48. Kubernetes Probes

Example:

`livenessProbe`

`readinessProbe`

`startupProbe`

StartupProbe er sérstaklega gagnleg fyrir services sem þurfa lengri initialization.

49. Resource Management

Every workload skal skilgreina:

`CPU request`

`CPU limit`

`memory request`

`memory limit`

Ekki leyfa unlimited workloads í production.

50. Horizontal Scaling

API:

HPA

byggt á:

`CPU`

`memory`

`request rate`

`latency`

Workers geta skalast eftir:

`queue depth`

51. Observability

Three pillars:

Logs

Metrics

Traces

Recommended:

OpenTelemetry

Prometheus
Grafana
Loki / equivalent

52. Logging

Central logging:

Application
↓
Log collector
↓
Central storage
Logs skulu innihalda:

timestamp
service
severity
request_id
trace_id
Ekki:

passwords
tokens
document contents
unnecessary PII

53. Audit Logging

Audit events skulu vera aðskilin frá normal application logs.

Audit:

append-only
tamper-resistant
retained
searchable
access controlled

54. Audit Event

Example:

```
{  
  "event": "DOCUMENT_DOWNLOADED",
```

```
"user_id": "...",  
"case_id": "...",  
"document_id": "...",  
"timestamp": "...",  
"result": "SUCCESS",  
"request_id": "..."  
}
```

55. Monitoring

Core metrics:

API latency
HTTP errors
request rate
database connections
database latency
queue depth
worker failures
storage usage
CPU
memory
disk

56. Alerting

Critical alerts:

API unavailable
database unavailable
replication failure
backup failure
queue backlog
certificate expiry
storage failure
high error rate
security event
Alerts skulu vera actionable.

57. Backup Strategy

Backup categories:

PostgreSQL
Object Storage
Configuration
Secrets metadata
Infrastructure state

58. PostgreSQL Backup

Recommended:

daily full backup
continuous WAL archiving
Retention example:

daily: 30 days
weekly: 12 weeks
monthly: 12 months
Exact retention verður policy-driven.

59. Object Storage Backup

Document storage skal hafa:

versioning
replication
immutability where required
Backup skal ekki aðeins vera:

copy of same storage bucket
því það verndar ekki gegn öllum failure scenarios.

60. Backup Encryption

Backups skulu vera encrypted.

Encryption keys skulu vera aðskildar frá application secrets.

61. Restore Testing

Backup er ekki talið gilt fyrr en restore hefur verið prófað.

Minimum:

monthly restore test
Critical production system:

quarterly full DR test

62. RPO

Initial target:

$RPO \leq 15 \text{ minutes}$
fyrir PostgreSQL.

Þetta skal staðfest með actual backup/replication architecture.

63. RTO

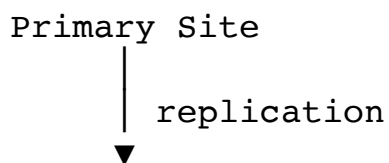
Initial target:

$RTO \leq 4 \text{ hours}$
fyrir full production recovery.

HA incident ætti að vera mun hraðari.

64. Disaster Recovery

DR architecture:



DR Site
DR site skal hafa:

database replica
object storage replica
infrastructure definitions
backup access

65. DR Principle

DR documentation skal innihalda:

who

does what
in what order
using which commands
ekki aðeins:

"restore from backup"

66. Business Continuity

Ef ILCMS er unavailable skal organization geta:

- identify incident
- activate DR
- continue critical legal work
- recover records
- document manual activities
- reconcile changes after recovery

67. Disaster Recovery Runbook

Required:

Detect

↓

Declare incident

↓

Assess

↓

Freeze changes

↓

Activate DR

↓

Restore/Promote DB

↓

Restore object storage

↓

Deploy application

↓

Validate

↓

Open service

↓

Monitor

↓

Post-incident review

68. Security Monitoring

Monitor:

failed logins
privilege changes
unusual downloads
bulk exports
permission changes
admin actions
suspicious API traffic

69. SIEM Integration

Enterprise deployment should support forwarding security events to:

SIEM

Examples:

Microsoft Sentinel

Splunk

Elastic Security

Architecture skal vera vendor-neutral.

70. Data Retention

Retention policies skulu vera configurable.

Examples:

case records

documents

audit logs

system logs

backups

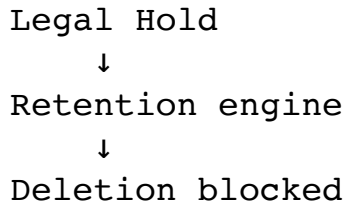
AI outputs

Retention er legal/policy concern og má ekki hard-code-a í frontend.

71. Legal Hold

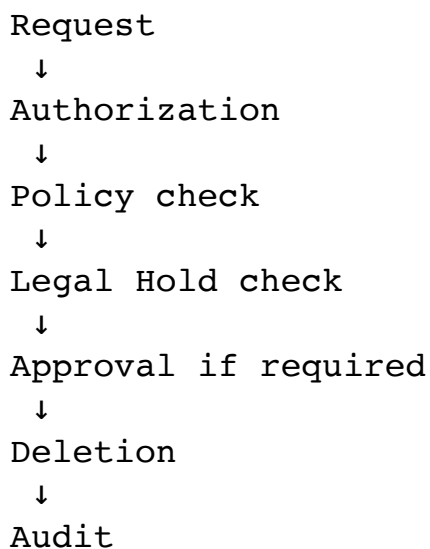
Legal hold skal override-a normal deletion/retention rules þar sem policy krefst þess.

Architecture:



72. Data Deletion

Deletion skal vera controlled workflow.



73. Object Deletion

Document deletion skal ekki þýða einfaldlega:

`DELETE FROM documents`

Database metadata, object storage og audit state verða að vera samræmd.

74. Export

System skal styðja controlled export:



Select records

↓

Authorization

↓

Generate export

↓

Audit

↓

Download

Exports skulu hafa expiration.

75. Bulk Export Security

Bulk export skal:

require explicit permission

be audited

be rate limited

possibly require approval

76. Production Access

Developers skulu ekki hafa permanent production admin access.

Preferred:

JIT access

MFA

audit

time-limited privileges

77. Kubernetes Access

RBAC:

Developer

Operator

Security

Database Administrator

Platform Administrator

Read Only

Least privilege skal gilda.

78. Break-Glass Access

Emergency access skal vera til staðar.

Break-glass:

high privilege
time limited
strong authentication
fully audited
post-incident review

79. MFA

MFA required fyrir:

administration
production access
security functions
privileged users

80. Infrastructure Secrets

Secrets:

database passwords
API keys
TLS keys
OAuth credentials
AI provider credentials
storage credentials
eru managed centrally.

81. DNS

Production:

app.example.is
api.example.is
Internal:

postgres.internal
rabbitmq.internal
redis.internal

Exact domain TBD.

82. TLS Certificate Management

Automated renewal preferred.

Possible:

`cert-manager`

`ACME`

`enterprise PKI`

83. Dependency Management

Dependencies skulu:

`pinned`

`regularly updated`

`scanned`

`tested`

Automated dependency update PRs preferred.

84. Patch Management

Critical security vulnerabilities:

`prioritized immediately`

Normal updates:

`monthly release cycle`

par sem operationally feasible.

85. Production Change Management

Production changes skulu hafa:

`change ID`

`reason`

`risk`

`approval`

`deployment version`

`rollback plan`

`result`

86. Rollback

Every deployment skal hafa rollback strategy.

Application:

previous image

Infrastructure:

previous IaC revision

Database:

forward-compatible migration

Database rollback má ekki byggjast blindandi á down migration.

87. Capacity Planning

Monitor:

CPU

RAM

database size

document storage

queue throughput

API traffic

Forecast:

3 months

6 months

12 months

88. Initial Production Sizing

Illustrative baseline:

Kubernetes:

3 worker nodes

API:

2–3 replicas

Worker:

2 replicas

Frontend:
2 replicas

PostgreSQL:
2+ nodes

Redis:
HA if required

RabbitMQ:
3 nodes for HA

Object Storage:
redundant
Þetta er baseline, ekki final sizing.

89. High Availability

No single application pod shall be a single point of failure.

Critical services skulu hafa:

`replicas >= 2`
Database og messaging skulu hafa appropriate HA architecture.

90. Availability Target

Initial target:

99.9% monthly availability
Potential future target:

99.95%

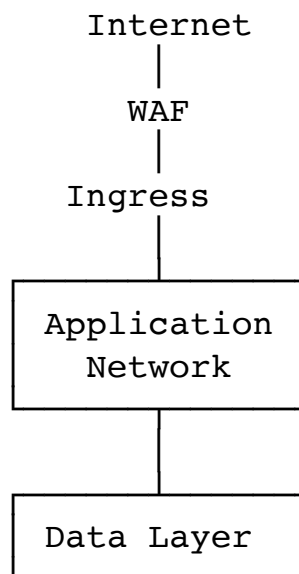
91. Failure Domains

Production architecture skal dreifa critical workloads yfir:

Kubernetes nodes
availability zones / failure domains
storage replicas
þar sem infrastructure leyfir.

92. Security Architecture

Overall:



Default:

deny

Allow only required communication.

93. Zero Trust Principle

Every request skal:

authenticate

authorize

validate

audit where appropriate

Internal network location er ekki nægjanlegt trust signal.

94. Infrastructure Security Boundary

User

↓

Identity Provider

↓

Frontend

↓

API

↓

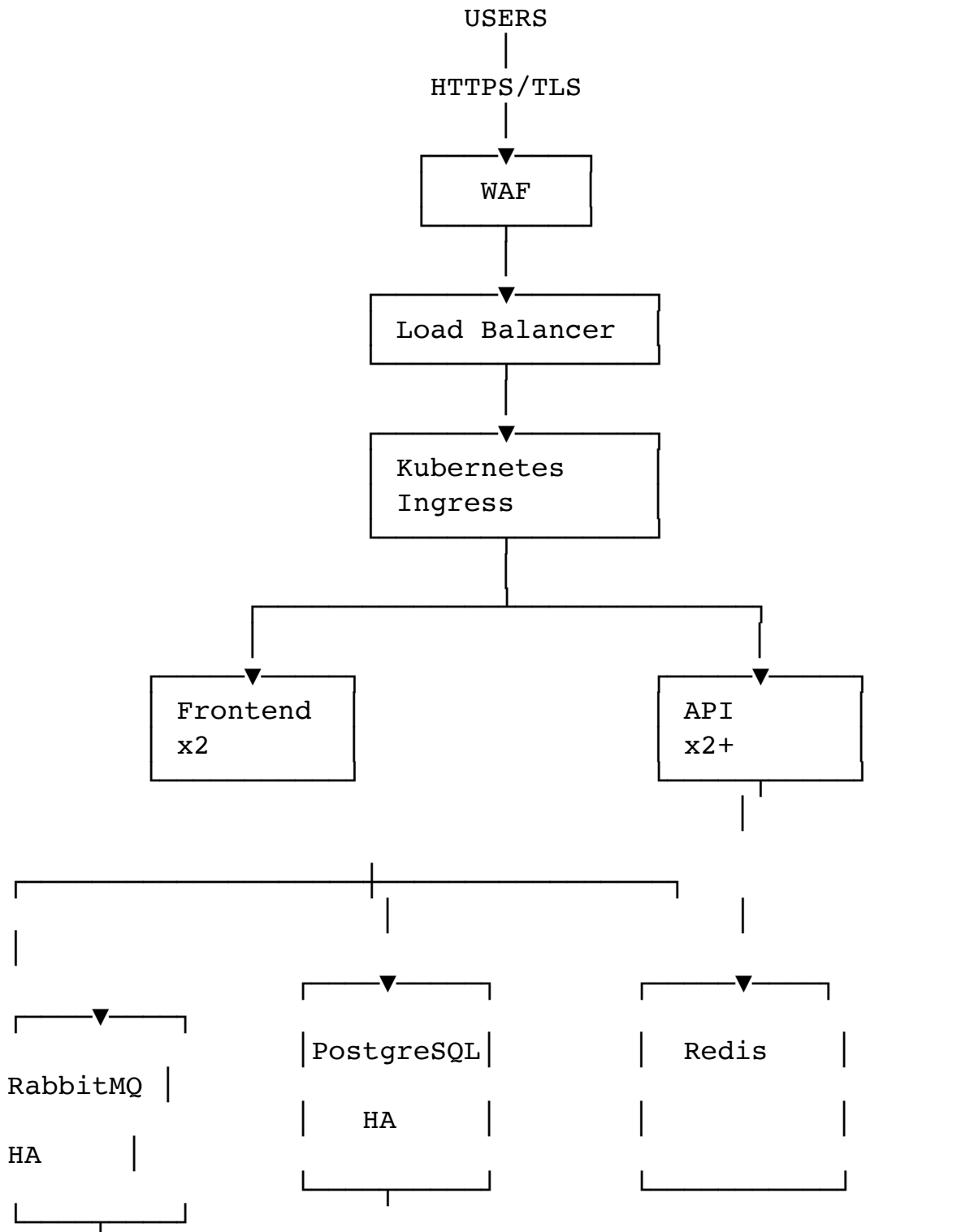
Authorization

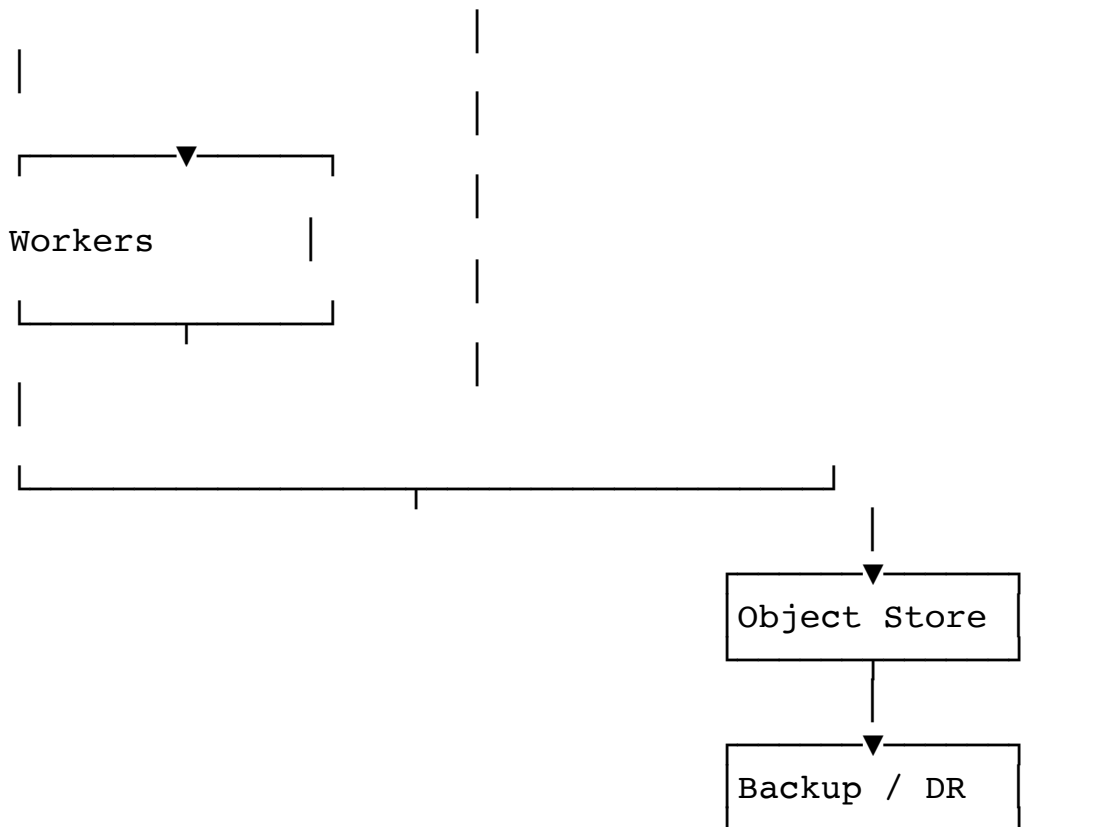
↓

Data

Frontend má aldreí bypass-a API authorization.

95. Production Architecture — Complete





96. Development Platform

Recommended developer environment:

Docker

Docker Compose

Services:

frontend

api

worker

postgres

redis

rabbitmq

object-storage

Developer skal geta keyrt major part kerfisins locally.

97. Staging

Staging skal vera eins production-like og practical.

Include:

same containers

same deployment mechanism
same database engine
same object storage semantics
same authentication flow

98. Production Promotion

Artifact promotion:

Build once

↓

Test

↓

Staging

↓

Production

Ekki rebuild-a application image sérstaklega fyrir production.

99. Immutable Infrastructure

Production server/container changes skulu ekki vera:

ssh

vim

restart

sem normal procedure.

Breyting:

Git

↓

CI

↓

Artifact

↓

Deployment

100. Infrastructure Definition of Done

Infrastructure v1.0 telst tilbúið þegar:

- Kubernetes cluster available
- namespaces configured

- ingress configured
- TLS automated
- frontend deployed
- API deployed
- workers deployed
- PostgreSQL HA configured
- Redis configured
- RabbitMQ HA configured
- object storage configured
- Vault/secrets management configured
- CI/CD implemented
- GitOps implemented
- image scanning implemented
- SBOM generated
- centralized logging configured
- metrics configured
- tracing configured
- audit pipeline configured
- backups automated
- restore tested
- DR documented
- RBAC configured
- MFA configured
- network policies configured
- vulnerability scanning configured
- monitoring alerts configured
- RPO/RTO tested
- production deployment documented

101. Recommended Technology Baseline

Layer	Technology
Frontend	React + TypeScript
API	FastAPI
Database	PostgreSQL
Cache	Redis
Messaging	RabbitMQ
Object Storage	S3-compatible
Containers	Docker
Orchestration	Kubernetes
IaC	Terraform
Kubernetes Packaging	Helm
GitOps	Argo CD
Secrets	HashiCorp Vault
Metrics	Prometheus
Dashboards	Grafana
Tracing	OpenTelemetry
Logging	Loki / equivalent
CI/CD	GitLab CI / equivalent
Registry	Harbor / private registry
AI	Pluggable local/private provider

102. Key Architecture Decisions

ADR-001

Kubernetes

Selected as production orchestration platform.

ADR-002

PostgreSQL

Primary system of record.

ADR-003

Object Storage

Binary legal documents stored outside relational DB.

ADR-004

RabbitMQ

Asynchronous processing and workflow messaging.

ADR-005

Vault

Centralized secret management.

ADR-006

OpenAPI

Contract between frontend and backend.

ADR-007

GitOps

Production infrastructure changes driven from Git.

ADR-008

Private AI

AI architecture must support local/private deployment.

ADR-009

Immutable deployments

No normal manual production modification.

ADR-010

Backup + tested restore

Backup is not considered valid until recovery is demonstrated.

103. Overall Architecture Status

With v1.0, ILCMS now has the following architecture layers:

```
v0.1  System Requirements
v0.2  Data Model
v0.3  PostgreSQL Schema & API
v0.4  Workflow & Security
v0.5  [Application/Domain Specification]
v0.6  Application Architecture
v0.7  [Implementation Specification]
v0.8  [Security / Operational foundation]
v0.9  Frontend Architecture & UX
v1.0  Infrastructure & Production Architecture
```

The project has now moved from:

Concept

to:

Implementable System Architecture

104. Next Step — v1.1

The next specification should move from architecture into **actual implementation contracts**.

Recommended:

Technical Specification v1.1 — DevOps, CI/CD & Kubernetes Implementation Specification

It should define the actual repository and deployment structure:

```
ilcms/
├── frontend/
├── backend/
├── worker/
├── infrastructure/
├── helm/
├── terraform/
├── gitops/
├── migrations/
├── docs/
└── tests/
```

and then specify:

- Dockerfiles

- docker-compose.yml
- Helm chart
- Kubernetes manifests
- namespaces
- RBAC
- NetworkPolicies
- Secrets/Vault integration
- PostgreSQL deployment
- RabbitMQ deployment
- Redis deployment
- object storage
- Ingress
- cert-manager
- Prometheus/Grafana
- CI pipeline
- CD pipeline
- Argo CD
- staging deployment
- production deployment
- database migration pipeline
- backup jobs
- restore jobs
- disaster recovery automation

v1.1 verður því fyrsta specification-ið sem hægt er að nota nánast beint sem blueprint fyrir repository, Kubernetes cluster og CI/CD pipeline.

Þetta er mikilvægur áfangi: með **v1.0** erum við ekki lengur bara að hanna forritið heldur **heilt production system**. Næsta skref, v1.1, getur því orðið mjög hagnýtt og nálgast raunverulega **Dockerfile**, Helm charts, Kubernetes manifests, Terraform og CI/CD uppsetningu.